

판다스 (Pandas) (4)

2026-1학기

ICT융합학부 컴퓨터공학트랙

김대환

목차

1. 벡터화된 문자열 연산
2. 시계열 다루기

벡터화된 문자열 연산

벡터화된 문자열 연산 (1)

- Pandas 문자열 연산 소개

- (예시) 전형적인 벡터화 연산

```
import numpy as np
x = np.array([2, 3, 5, 7, 11, 13])
x * 2
```

```
array([ 4,  6, 10, 14, 22, 26])
```

- NumPy에서의 문자열 배열 접근

```
data = ['peter', 'Paul', 'MARY', 'gUIDO']
[s.capitalize() for s in data]
```

```
['Peter', 'Paul', 'Mary', 'Guido']
```

```
data = ['peter', 'Paul', None, 'MARY', 'gUIDO']
[s.capitalize() for s in data]
```

```
-----
AttributeError                                Traceback (most recent call last)
Cell In[3], line 2
      1 data = ['peter', 'Paul', None, 'MARY', 'gUIDO']
----> 2 [s.capitalize() for s in data]
```

```
AttributeError: 'NoneType' object has no attribute 'capitalize'
```

Fix Code

벡터화된 문자열 연산 (2)

■ Pandas 문자열 메서드 목록

len()	lower()	translate()	islower()
ljust()	upper()	startswith()	isupper()
rjust()	find()	endswith()	isnumeric()
center()	rfind()	isalnum()	isdecimal()
zfill()	index()	isalpha()	split()
strip()	rindex()	isdigit()	rsplit()
rstrip()	capitalize()	isspace()	partition()
lstrip()	swapcase()	istitle()	rpartition()

- (예시들)

```
monte = pd.Series(['Graham Chapman', 'John Cleese', 'Terry Gilliam',  
                  'Eric Idle', 'Terry Jones', 'Michael Palin'])
```

monte.str.lower()

```
0    graham chapman  
1      john cleese  
2    terry gilliam  
3       eric idle  
4    terry jones  
5  michael palin  
dtype: object
```

monte.str.len()

```
0     14  
1     11  
2     13  
3      9  
4     11  
5     13  
dtype: int64
```

monte.str.split()

```
0    [Graham, Chapman]  
1    [John, Cleese]  
2    [Terry, Gilliam]  
3    [Eric, Idle]  
4    [Terry, Jones]  
5    [Michael, Palin]  
dtype: object
```

monte.str.startswith('T')

```
0    False  
1    False  
2     True  
3    False  
4     True  
5    False  
dtype: bool
```

벡터화된 문자열 연산 (3)

정규 표현식을 활용하는 메소드

- 정규 표현식을 활용하고, re 모듈의 API 규칙을 따라는 메소드 존재

Method	Description
<code>match()</code>	Call <code>re.match()</code> on each element, returning a boolean.
<code>extract()</code>	Call <code>re.match()</code> on each element, returning matched groups as strings.
<code>findall()</code>	Call <code>re.findall()</code> on each element
<code>replace()</code>	Replace occurrences of pattern with some other string
<code>contains()</code>	Call <code>re.search()</code> on each element, returning a boolean
<code>count()</code>	Count occurrences of pattern
<code>split()</code>	Equivalent to <code>str.split()</code> , but accepts regexps
<code>rsplit()</code>	Equivalent to <code>str.rsplit()</code> , but accepts regexps

예시들

```
monte.str.extract('([A-Za-z]+)', expand=False)
```

```
0    Graham
1      John
2    Terry
3      Eric
4    Terry
5   Michael
dtype: object
```

- [A-Za-z]:** 영어 대소문자 1개
- +:** 1개 이상 반복
- ():** 그룹

```
monte.str.findall(r'^([AEIOU]).*([aeiou])$')
```

```
0    [Graham Chapman]
1                []
2    [Terry Gilliam]
3                []
4    [Terry Jones]
5    [Michael Palin]
dtype: object
```

- ^:** 문자열의 시작
- [^AEIOU]:** 대문자 모음(A,E,I,O,U)이 아닌 문자
- .*:** 아무 문자 0개 이상
- [^aeiou]:** 소문자 모음(a,e,i,o,u)이 아닌 문자
- \$:** 문자열의 끝

벡터화된 문자열 연산 (4)

■ 기타 메소드

Method	Description
<code>get()</code>	Index each element
<code>slice()</code>	Slice each element
<code>slice_replace()</code>	Replace slice in each element with passed value
<code>cat()</code>	Concatenate strings
<code>repeat()</code>	Repeat values
<code>normalize()</code>	Return Unicode form of string
<code>pad()</code>	Add whitespace to left, right, or both sides of strings
<code>wrap()</code>	Split long strings into lines with length less than a given width
<code>join()</code>	Join strings in each element of the Series with passed separator
<code>get_dummies()</code>	extract dummy variables as a dataframe

■ 벡터화된 항목의 접근 및 슬라이싱

```
monte.str[0:3]
```

```
0    Gra
1    Joh
2    Ter
3    Eri
4    Ter
5    Mic
dtype: object
```

```
monte.str.split().str.get(-1)
```

```
0    Chapman
1    Cleese
2    Gilliam
3    Idle
4    Jones
5    Palin
dtype: object
```

벡터화된 문자열 연산 (5)

- 기타 메소드 (2)

- 지시 변수

- get_dummies(): 범주형 데이터를 숫자형 (0/1)으로 바꾸는 대표적인 인코딩 함수

```
full_monte = pd.DataFrame({'name': monte,  
                           'info': ['B|C|D', 'B|D', 'A|C',  
                                    'B|D', 'B|C', 'B|C|D']})  
full_monte
```

	name	info
0	Graham Chapman	B C D
1	John Cleese	B D
2	Terry Gilliam	A C
3	Eric Idle	B D
4	Terry Jones	B C
5	Michael Palin	B C D

```
full_monte['info'].str.get_dummies('|')
```

	A	B	C	D
0	0	1	1	1
1	0	1	0	1
2	1	0	1	0
3	0	1	0	1
4	0	1	1	0
5	0	1	1	1

시계열 다루기

시계열 다루기 (1)

- **Pandas 날짜와 시간 데이터 종류**

- **타임스탬프**: 특정 시점을 말함 (예. 2015년 6월 4일 오전 7시)
- **시간 간격**: 특정 시작점과 종료점 사이의 시간의 길이 (예. 2015년)
- **기간**: 각 간격이 일정하고 서로 겹치지 않는 특별한 경우의 시간 간격 (예. 하루를 구성하는 24시간이라는 기간)
- **시간 델타 (Time delta)나 지속 기간 (Duration)**: 정확한 시간 길이 (예. 22.56초의 시간)

시계열 다루기 (2)

■ 파이썬에서의 날짜와 시간 (1)

■ 기본 파이썬 날짜와 시간: datetime과 dateutil

```
from datetime import datetime
datetime(year=2015, month=7, day=4)
```

```
datetime.datetime(2015, 7, 4, 0, 0)
```

```
from dateutil import parser
date = parser.parse("4th of July, 2015")
date
```

```
datetime.datetime(2015, 7, 4, 0, 0)
```

```
date.strftime('%A')
```

```
'Saturday'
```

- ➔ 큰 규모의 날짜와 시간 배열 작업 시 느림
- ➔ 인코딩된 날짜의 타입 지정 배열이 파이썬 datetime 객체의 리스트보다 나옴 !!

코드	분류	의미	예시
%Y	날짜	연도 (4자리)	2026
%y	날짜	연도 (2자리)	26
%m	날짜	월 (01~12)	04
%B	날짜	월 이름 (전체)	April
%b	날짜	월 이름 (축약)	Apr
%d	날짜	일 (01~31)	06
%A	요일	요일 (전체)	Monday
%a	요일	요일 (축약)	Mon
%w	요일	요일 숫자 (일=0)	1
%H	시간	시 (24시간)	14
%I	시간	시 (12시간)	02
%p	시간	AM / PM	PM
%M	시간	분	30
%S	시간	초	45

시계열 다루기 (2)

- 파이썬에서의 날짜와 시간 (2)

- 타입이 지정된 시간 배열: NumPy의 datetime64
 - NumPy에 몇 가지 기본 시계열 데이터 타입 추가
 - **datetime64**
 - 날짜를 64비트 정수로 인코딩해 날짜 배열을 간결하게 표현
 - 구체적 입력 형식이 필요함

```
import numpy as np
date = np.array('2015-07-04', dtype=np.datetime64)
date
```

```
array('2015-07-04', dtype='datetime64[D]')
```

```
date + np.arange(12)
```

```
array(['2015-07-04', '2015-07-05', '2015-07-06', '2015-07-07',
      '2015-07-08', '2015-07-09', '2015-07-10', '2015-07-11',
      '2015-07-12', '2015-07-13', '2015-07-14', '2015-07-15'],
      dtype='datetime64[D]')
```

시계열 다루기 (3)

■ 파이썬에서의 날짜와 시간 (3)

■ datetime64와 timedelta64

- 기본 시간 단위 기반으로 만들어졌음
- datetime64 객체: 64비트 정밀도로 제한되기에, 표현 가능한 시간 범위는 선택한 시간 단위 (예: 초, 밀리초 등)에 따라 달라짐 → 시간의 정밀도 (얼마나 세밀하게 표현할지)와 표현 가능한 최대 시간 범위 사이에 Trade-off 존재

```
np.datetime64('2015-07-04')
```

```
numpy.datetime64('2015-07-04')
```

```
np.datetime64('2015-07-04 12:00')
```

```
numpy.datetime64('2015-07-04T12:00')
```

```
np.datetime64('2015-07-04 12:59:59.50', 'ns')
```

```
numpy.datetime64('2015-07-04T12:59:59.500000000')
```

시계열 다루기 (4)

- 파이썬에서의 날짜와 시간 (4)
 - datetime64가 인코딩할 수 있는 상대적/절대적 시간 범위

Code	Meaning	Time span (relative)	Time span (absolute)
Y	Year	$\pm 9.2e18$ years	[9.2e18 BC, 9.2e18 AD]
M	Month	$\pm 7.6e17$ years	[7.6e17 BC, 7.6e17 AD]
W	Week	$\pm 1.7e17$ years	[1.7e17 BC, 1.7e17 AD]
D	Day	$\pm 2.5e16$ years	[2.5e16 BC, 2.5e16 AD]
h	Hour	$\pm 1.0e15$ years	[1.0e15 BC, 1.0e15 AD]
m	Minute	$\pm 1.7e13$ years	[1.7e13 BC, 1.7e13 AD]
s	Second	$\pm 2.9e12$ years	[2.9e9 BC, 2.9e9 AD]
ms	Millisecond	$\pm 2.9e9$ years	[2.9e6 BC, 2.9e6 AD]
us	Microsecond	$\pm 2.9e6$ years	[290301 BC, 294241 AD]
ns	Nanosecond	± 292 years	[1678 AD, 2262 AD]
ps	Picosecond	± 106 days	[1969 AD, 1970 AD]
fs	Femtosecond	± 2.6 hours	[1969 AD, 1970 AD]
as	Attosecond	± 9.2 seconds	[1969 AD, 1970 AD]

시계열 다루기 (5)

■ Pandas에서의 날짜와 시간

- Series나 DataFrame 데이터에 인덱스를 지정하는 사용할 수 있는 (Timestamp 객체 그룹으로부터) DateTimeIndex 구성 가능

```
import pandas as pd
date = pd.to_datetime("4th of July, 2015")
date
```

```
Timestamp('2015-07-04 00:00:00')
```

```
date.strftime('%A')
```

```
'Saturday'
```

```
date + pd.to_timedelta(np.arange(12), 'D')
```

```
DatetimeIndex(['2015-07-04', '2015-07-05', '2015-07-06', '2015-07-07',
               '2015-07-08', '2015-07-09', '2015-07-10', '2015-07-11',
               '2015-07-12', '2015-07-13', '2015-07-14', '2015-07-15'],
              dtype='datetime64[ns]', freq=None)
```

시계열 다루기 (6)

- Pandas 시계열: 시간으로 인덱싱하기
 - 타임스탬프로 데이터를 인덱싱할 때 유용

```
index = pd.DatetimeIndex(['2014-07-04', '2014-08-04',  
                          '2015-07-04', '2015-08-04'])  
data = pd.Series([0, 1, 2, 3], index=index)  
data
```

```
2014-07-04    0  
2014-08-04    1  
2015-07-04    2  
2015-08-04    3  
dtype: int64
```

```
data['2014-07-04':'2015-07-04']
```

```
2014-07-04    0  
2014-08-04    1  
2015-07-04    2  
dtype: int64
```

```
data['2015']
```

```
2015-07-04    2  
2015-08-04    3  
dtype: int64
```


시계열 다루기 (7)

■ Pandas 시계열 데이터 구조

- **타임스탬프 (Time stamp)**: Timestamp 타입을 제공. Numpy.datetime64 데이터 타입을 기반으로 함. 인덱스 구조는 **DatetimeIndex** 임
- **기간 (Time period)**: Period 타입을 제공. Numpy.datetime64를 기반으로 고정 주파수 간격을 인코딩 함. 인덱스 구조는 **PeriodIndex** 임
- **시간 델타 및 지속 시간**: TimeDelta 타입을 제공. Numpy.timedelta64를 기반으로 함. 인덱스 구조는 **TimedeltaIndex** 임

■ `pd.to_datetime ( )` 함수

- 단일 날짜를 전달하면 Timestamp를 생성하고, 일련의 날짜를 전달하면 DatetimeIndex를 생성

```
dates = pd.to_datetime([datetime(2015, 7, 3), '4th of July, 2015',  
                        '2015-Jul-6', '07-07-2015', '20150708'])
```

```
dates
```

```
DatetimeIndex(['2015-07-03', '2015-07-04', '2015-07-06', '2015-07-07',  
              '2015-07-08'],  
              dtype='datetime64[ns]', freq=None)
```

```
dates.to_period('D')
```

```
PeriodIndex(['2015-07-03', '2015-07-04', '2015-07-06', '2015-07-07',  
            '2015-07-08'],  
            dtype='period[D]')
```

```
dates - dates[0]
```

```
TimedeltaIndex(['0 days', '1 days', '3 days', '4 days', '5 days'], dtype='timedelta64[ns]', freq=None)
```

시계열 다루기 (8)

■ 정규 시퀀스: `pd.date_range()`

- 정규 날짜 시퀀스를 편리하게 만들 수 있도록 몇 가지 함수 제공
- `pd.date_range()`: 타임 스탬프 → 시작일, 종료일, 선택적 주기 코드를 받아 정규 날짜 시퀀스 생성 (기본 주기는 하루)
- `pd.period_range()`: 기간
- `pd.timedelta_range()`: 시간 델타

```
pd.date_range('2015-07-03', '2015-07-10')
```

```
DatetimeIndex(['2015-07-03', '2015-07-04', '2015-07-05', '2015-07-06',  
               '2015-07-07', '2015-07-08', '2015-07-09', '2015-07-10'],  
              dtype='datetime64[ns]', freq='D')
```

```
pd.date_range('2015-07-03', periods=8)
```

```
DatetimeIndex(['2015-07-03', '2015-07-04', '2015-07-05', '2015-07-06',  
               '2015-07-07', '2015-07-08', '2015-07-09', '2015-07-10'],  
              dtype='datetime64[ns]', freq='D')
```

```
pd.date_range('2015-07-03', periods=8, freq='h')
```

```
DatetimeIndex(['2015-07-03 00:00:00', '2015-07-03 01:00:00',  
               '2015-07-03 02:00:00', '2015-07-03 03:00:00',  
               '2015-07-03 04:00:00', '2015-07-03 05:00:00',  
               '2015-07-03 06:00:00', '2015-07-03 07:00:00'],  
              dtype='datetime64[ns]', freq='h')
```

```
pd.period_range('2015-07', periods=8, freq='M')
```

```
PeriodIndex(['2015-07', '2015-08', '2015-09', '2015-10', '2015-11', '2015-12',  
             '2016-01', '2016-02'],  
            dtype='period[M]')
```

```
pd.timedelta_range(0, periods=10, freq='h')
```

```
TimedeltaIndex(['0 days 00:00:00', '0 days 01:00:00', '0 days 02:00:00',  
                '0 days 03:00:00', '0 days 04:00:00', '0 days 05:00:00',  
                '0 days 06:00:00', '0 days 07:00:00', '0 days 08:00:00',  
                '0 days 09:00:00'],  
               dtype='timedelta64[ns]', freq='h')
```

시계열 다루기 (9)

■ 주기와 오프셋

- Pandas 시계열 도구는 주기나 날짜 오프셋 개념을 기반으로 함

CodeDescription CodeDescription

D	Calendar day	B	Business day
W	Weekly		
M	Month end	BM	Business month end
Q	Quarter end	BQ	Business quarter end
A	Year end	BA	Business year end
H	Hours	BH	Business hours
T	Minutes		
S	Seconds		
L	Milliseconds		
U	Microseconds		
N	nanoseconds		

→ 영업일 기준으로 산정

→ 월, 분기, 연 단위의 주기는
모두 지정한 기간의 종료시
점으로 표시

CodeDescription CodeDescription

MS	Month start	BMS	Business month start
QS	Quarter start	BQS	Business quarter start
AS	Year start	BAS	Business year start

- Q-JAN, BQ-FEB, QS-MAR, BQS-APR, etc.
- A-JAN, BA-FEB, AS-MAR, BAS-APR, etc.
- W-SUN, W-MON, W-TUE, W-WED, etc.

```
pd.timedelta_range(0, periods=9, freq="2h30min")
```

```
TimedeltaIndex(['0 days 00:00:00', '0 days 02:30:00', '0 days 05:00:00',  
                '0 days 07:30:00', '0 days 10:00:00', '0 days 12:30:00',  
                '0 days 15:00:00', '0 days 17:30:00', '0 days 20:00:00'],  
               dtype='timedelta64[ns]', freq='150min')
```

```
pd.date_range('2026-04-01', periods=3, freq='W-MON')
```

```
DatetimeIndex(['2026-04-06', '2026-04-13', '2026-04-20'], dtype='datetime64[ns]', freq='W-MON')
```

```
pd.date_range('2026-01-01', periods=6, freq='QS-MAR')
```

```
DatetimeIndex(['2026-03-01', '2026-06-01', '2026-09-01', '2026-12-01',  
                '2027-03-01', '2027-06-01'],  
               dtype='datetime64[ns]', freq='QS-MAR')
```

→ 접미사 S를 추가하면 종료기
아니라 시작 시점으로 표시

→ Q (분기), A (연간), W (주간)

→ Q-JAN: 1월을 기준으로 분기 끝

→ QS-MAR: 3월을 기준으로 분기 시작

→ A-JAN: 1월이 연말

→ W-SUN: 일요일 기준

(필요이유) 기관마다 회계연도가 다르기에 사용

시계열 다루기 (10)

리샘플링, 시프팅, 윈도우

- Pandas는 시계열 전용 연산을 제공
- 주로 금융 환경에서 개발되어 왔기에, 금융 데이터에 특화된 전용 도구를 포함

```
import yfinance as yf
```

```
goog = yf.download('GOOG', start='2004-01-01', end='2016-01-01')  
goog.head()
```

[*****100%*****] 1 of 1 completed

Price	Close	High	Low	Open	Volume
Ticker	GOOG	GOOG	GOOG	GOOG	GOOG
Date					
2004-08-19	2.478782	2.570680	2.370580	2.470382	897427216
2004-08-20	2.675672	2.694694	2.482735	2.495333	458857488
2004-08-23	2.702599	2.803390	2.693952	2.735949	366857939
2004-08-24	2.590690	2.756947	2.558575	2.748054	306396159
2004-08-25	2.618605	2.668014	2.566233	2.592913	184645512

```
goog = goog['Close']
```

```
%matplotlib inline  
import matplotlib.pyplot as plt  
import seaborn; seaborn.set()
```

```
goog.plot();  
plt.show()
```



시계열 다루기 (10)

리샘플링, 시프팅, 윈도우 (3)

리샘플링 및 주기 변경

- 높거나 낮은 주기로 표본을 다시 추출함
- `resample()`: 데이터를 새로운 시간 단위로 묶어서 계산 (집계)
- `asfreq()`: 데이터를 선택함

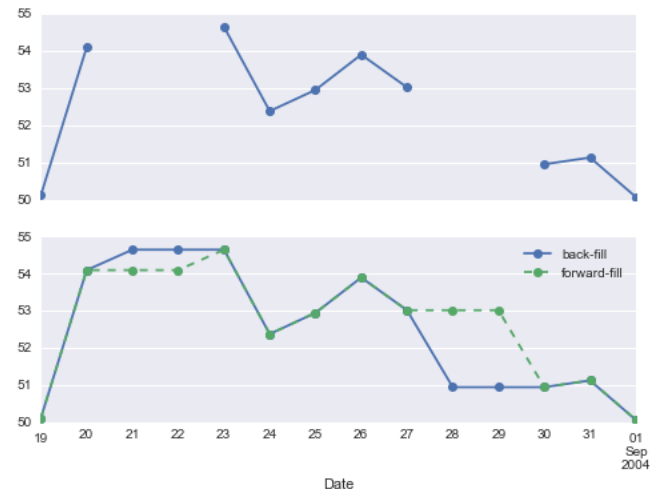
```
goog.plot(alpha=0.5, style='-')
goog.resample('BA').mean().plot(style=':')
goog.asfreq('BA').plot(style='--');
plt.legend(['input', 'resample', 'asfreq'],
           loc='upper left');
plt.show()
```



```
fig, ax = plt.subplots(2, sharex=True)
data = goog.iloc[:10]

data.asfreq('D').plot(ax=ax[0], marker='o')

data.asfreq('D', method='bfill').plot(ax=ax[1], style='-o')
data.asfreq('D', method='ffill').plot(ax=ax[1], style='--o')
ax[1].legend(["back-fill", "forward-fill"]);
```



시계열 다루기 (11)

- 리샘플링, 시프팅, 윈도우 (4)
 - 시간 이동 (Time-shift)
 - 데이터의 시간 이동
 - shift 메소드: 주어진 항목 수 만큼 데이터를 이동하는데 사용



```
import pandas as pd
import matplotlib.pyplot as plt

fig, ax = plt.subplots(3, sharey=True)

# apply a frequency to the data
goog = goog.asfreq('D', method='pad')

# 1) 원본
goog.plot(ax=ax[0])

# 2) 값 이동 (기존 shift)
goog.shift(900).plot(ax=ax[1])

# 3) 시간 이동 (기존 tshift → 대체)
goog.shift(900, freq='D').plot(ax=ax[2])

# Legends and annotations
local_max = pd.to_datetime('2007-11-05')
offset = pd.Timedelta(900, 'D')

ax[0].legend(['input'], loc=2)
ax[0].get_xticklabels()[2].set(weight='heavy', color='red')
ax[0].axvline(local_max, alpha=0.3, color='red')

ax[1].legend(['shift(900)'], loc=2)
ax[1].get_xticklabels()[2].set(weight='heavy', color='red')
ax[1].axvline(local_max + offset, alpha=0.3, color='red')

ax[2].legend(['shift(900, freq="D")'], loc=2)
ax[2].get_xticklabels()[1].set(weight='heavy', color='red')
ax[2].axvline(local_max + offset, alpha=0.3, color='red')

plt.show()
```

시계열 다루기 (12)

- 리샘플링, 시프팅, 윈도우 (5)
 - 롤링 윈도우 (Rolling windows)
 - rolling() : 일정 구간을 잘라서 계산 (예, 이동 평균 등)

```
rolling = goog.rolling(365, center=True)

data = pd.DataFrame({'input': goog,
                     'one-year rolling_mean': rolling.mean(),
                     'one-year rolling_std': rolling.std()})

ax = data.plot(style=['-', '--', ':'])
ax.lines[0].set_alpha(0.3)
```



Q & A
